COLLECTION GÉNIE LOGICIEL · LIVRE 1

Python

par la pratique

Les fondamentaux de la programmation

à pratiquer pas à pas, seul ou en formation

Génie logiciel DQP / DTS

Option développement d'application

Moudériko Mouansié

Formateur référent en développement d'applications

MINEFOP Cameroun

Édition ASSIIN

Association des Informaticiens Innovants

Première version · Partie 1

Python par la pratique

Livre 1 : les fondamentaux de la programmation, à pratiquer pas à pas dans l'interpréteur de commandes Python

Auteur : **Moudériko Mouansié**
Formateur référent en développement d’applications, MINEFOP Cameroun

Édition : **ASSIIN** (Association des Informaticiens Innovants)

Public : étudiants en Génie logiciel DQP / DTS, option développement d’application

Version de référence : **Python 3.15**. Première version, contenant l’avant-propos, la Partie 1 et ses travaux pratiques.

© Moudériko Mouansié, Édition ASSIIN. Tous droits réservés. Toute reproduction, même partielle, est soumise à l’autorisation de l’auteur.

Sommaire

Avant-propos 4

Partie 1 : Démarrer avec Python 7

Chapitre 1 : Comprendre la programmation et découvrir Python 8

Chapitre 2 : Installer Python et découvrir l'interpréteur de commandes 11

Chapitre 3 : Variables et types de données 18

Chapitre 4 : Opérateurs, saisie et affichage 24

Travaux pratiques de la Partie 1 31

TP 1 : Un convertisseur de durées, de températures et de monnaies 32

TP 2 : Un ticket de caisse 34

TP 3 : Le bulletin de notes d'un étudiant 36

Lexique et fiche mémo de la Partie 1 38

Avant-propos

À qui s'adresse ce livre ?

Ce livre est destiné aux étudiants de la filière **Génie logiciel** (DQP et DTS), option **développement d'application**. Il suppose seulement que vous savez utiliser un ordinateur : aucune connaissance préalable en programmation n'est nécessaire. Il a été écrit pour que vous puissiez **le lire et le comprendre seul**, à votre rythme, tout en restant un support de formation pour les séances encadrées par un formateur.

Ce que vous saurez faire à la fin du livre

- écrire, exécuter et corriger des programmes Python de taille moyenne ;
- manipuler les données avec les listes, dictionnaires et fichiers ;
- organiser votre code avec des fonctions, des modules et des classes ;
- rendre un programme fiable grâce à la gestion des erreurs et aux tests ;
- versionner votre travail avec Git et le livrer proprement.

Comment travailler avec ce livre

Programmer s'apprend en **pratiquant**, pas en lisant. Pour chaque chapitre, suivez toujours la même méthode :

- **Lisez** le chapitre en entier une première fois, sans toucher à l'ordinateur.
- **Reprenez** chaque exemple en le tapant vous-même dans l'interpréteur Python (l'invite de commandes), et vérifiez que vous obtenez le même résultat.
- **Expérimentez** : modifiez les exemples, essayez d'autres valeurs, provoquez des erreurs. C'est en cassant les choses qu'on les comprend.
- **Faites les exercices** à la fin du chapitre, puis répondez aux questions « Testez-vous ».
- En fin de partie, réalisez les **travaux pratiques** (TP), qui réunissent toutes les notions vues.

Prévoyez en moyenne une à deux séances de deux heures par chapitre.

Comment lire les exemples

Dans ce livre, nous travaillons dans l'**interpréteur de commandes Python**, lancé depuis l'invite de commandes (*cmd* sous Windows, le terminal sous Linux et macOS). Voici comment lire les exemples :

Ce que vous voyez	Ce que cela signifie
`>>> 2 + 3`	Vous tapez `2 + 3` après l'invite `>>>` (ne tapez pas les trois signes >), puis vous validez avec Entrée.
`5`	Une ligne sans `>>>` est la réponse de Python.
`C:\Users\etudiant>`	Invite de commandes de Windows (le nom du dossier peut être différent chez vous).
`# texte`	Un commentaire : Python l'ignore, il sert à expliquer.

Les rubriques du livre

Rubrique	Rôle
Objectifs	Ce que vous saurez faire à la fin du chapitre.
À vous de jouer	De petites manipulations à faire tout de suite dans l'interpréteur.
Points d'attention	Les pièges et les erreurs fréquentes des débutants.
À retenir	L'essentiel du chapitre, à relire avant les exercices.
Exercices	Des exercices de difficulté croissante (une à trois étoiles).
Testez-vous	Quelques questions pour vérifier que vous avez compris.
TP	Des travaux pratiques complets en fin de partie.

À propos des exercices

Les exercices de ce livre **ne sont pas corrigés** : ils sont conçus pour la formation, et les corrigés sont remis par le formateur. Cela ne doit pas vous bloquer, car **l'interpréteur est votre meilleur correcteur** : vous tapez votre solution, vous regardez le résultat et vous le comparez à ce qui est attendu. Quand c'est utile, la rubrique « Pour vérifier » vous donne le résultat à obtenir.

Que faire quand on est bloqué ?

- Relisez calmement le **message d'erreur** : il indique le type d'erreur et la ligne.
- Testez votre idée **par petits morceaux** dans l'interpréteur.
- Utilisez l'aide intégrée de Python avec `help()` (chapitre 2).
- Comparez avec un exemple du chapitre qui ressemble à votre problème.
- En formation, notez votre question et posez-la à votre formateur.

Plan général du livre

Partie	Contenu	Chapitres
1	Démarrer avec Python	1 à 4
2	Faire des choix et répéter des actions	5 à 7
3	Organiser les données	8 à 10
4	Structurer son code	11 à 13
5	Données et fiabilité	14 à 17
6	Programmation orientée objet	18 à 22
7	Qualité et outils du développeur	23 à 25
8	Projet final	26 et 27

Cette version

Ce volume contient l'avant-propos et la **Partie 1** (chapitres 1 à 4), avec ses trois travaux pratiques, un lexique et une fiche mémo. Les autres parties seront ajoutées progressivement. Les suites de la collection (bases de données, interfaces graphiques, Django, etc.) feront l'objet de livres de spécialisation distincts.

Pour aller plus loin

Ce livre est un contenu original. Pour approfondir ou varier les approches, vous pouvez consulter en complément les cours Python proposés sur OpenClassrooms ainsi que la documentation officielle sur python.org.

Partie 1
Démarrer avec Python

Chapitre 1 : Comprendre la programmation et découvrir Python

Objectifs du chapitre

- expliquer ce qu'est un programme et un algorithme ;
- situer Python parmi les langages de programmation ;
- citer les principaux usages de Python ;
- distinguer l'interpréteur interactif et le script.

1.1 Qu'est-ce qu'un programme ?

Un ordinateur sait exécuter des instructions très simples, mais il les exécute très vite et sans jamais se fatiguer. En revanche, il ne devine rien : il fait exactement ce qu'on lui demande, ni plus, ni moins. Un **programme** est une suite d'instructions écrites dans un langage que l'ordinateur sait traiter, et qui décrit comment accomplir une tâche.

On peut comparer un programme à une **recette de cuisine** : les ingrédients sont les données, les étapes de la recette sont les instructions, et le plat obtenu est le résultat. Si une étape est oubliée ou mal écrite, le plat est raté, et un programme aussi.

Avant d'écrire un programme, on réfléchit à la méthode à suivre. Cette méthode, décrite étape par étape et sans langage informatique, s'appelle un **algorithme**. Voici par exemple l'algorithme qui calcule le prix toutes taxes comprises (TTC) d'un article :

- demander le prix hors taxe (HT) ;
- demander le taux de TVA ;
- calculer le montant de la TVA : prix HT multiplié par le taux, divisé par 100 ;
- calculer le prix TTC : prix HT plus le montant de la TVA ;
- afficher le prix TTC.

L'algorithme, c'est l'*idée* ; le programme, c'est sa traduction dans un langage précis. Un bon développeur commence toujours par l'algorithme.

1.2 Les langages de programmation

L'ordinateur ne comprend réellement que le **langage machine**, fait de 0 et de 1. Écrire directement dans ce langage serait extrêmement pénible. On utilise donc des **langages de haut niveau** (Python, Java, C#, PHP, JavaScript...) qui ressemblent davantage au langage humain, et qui sont traduits pour la machine. Cette traduction se fait de deux façons principales :

	Langage compilé	Langage interprété
Principe	Le programme entier est traduit en un fichier exécutable avant d'être lancé.	Le programme est lu et traduit pendant son exécution, par un interpréteur.
Exemples	C, C++, Rust	Python, PHP, JavaScript
Avantage	Exécution souvent plus rapide.	Test immédiat, développement plus rapide.

Python est un langage **plutôt interprété** : vous tapez une instruction, l'interpréteur l'exécute aussitôt et vous voyez le résultat. C'est un atout énorme pour apprendre, car on peut essayer, se tromper et recommencer en quelques secondes.

1.3 Pourquoi apprendre Python ?

- **Il est lisible.** Sa syntaxe est claire et se rapproche du langage naturel, ce qui réduit le temps d'apprentissage.
- **Il est polyvalent.** On l'utilise pour les applications de bureau, le web (avec des outils comme Django, que nous verrons dans un livre de spécialisation), l'automatisation de tâches, l'analyse de données, l'intelligence artificielle...
- **Il dispose d'un écosystème immense.** Des milliers de bibliothèques gratuites existent pour presque tous les besoins.
- **Il a une grande communauté.** Documentation, forums et tutoriels sont abondants, ce qui facilite la résolution des problèmes.

Pour vous donner une idée, voici l'instruction qui affiche un message à l'écran. En Python, elle tient en une ligne :

```
print("Bonjour tout le monde")
```

Dans beaucoup d'autres langages, il faut plusieurs lignes de préparation pour obtenir le même résultat. Ce livre prend pour référence **Python 3.15**, publiée en octobre 2026.

> **Python 3.15 en bref**
>
> • **UTF-8 par défaut :** les accents et caractères spéciaux sont mieux gérés dans vos fichiers et vos programmes, quel que soit le système.
>
> • **Messages d'erreur améliorés et plus colorés** dans le terminal, ce qui facilite le diagnostic des erreurs.
>
> • **Nouveaux types intégrés** : *frozendict* (dictionnaire non modifiable) et *sentinel* (valeur repère unique). Nous les rencontrerons plus tard dans le livre.
>
> • **Outils de performance** : un nouveau profileur par échantillonnage et un compilateur JIT amélioré, utiles surtout pour les projets avancés.
>
> Tout ce que vous apprenez dans ce livre reste valable : les bases du langage sont stables d'une version à l'autre.

1.4 Deux façons de travailler avec Python

Il existe deux manières d'exécuter du code Python. Vous allez utiliser les deux, mais dans cet ordre :

	L'interpréteur interactif	Le script
Principe	Vous tapez une instruction, Python répond tout de suite.	Vous écrivez plusieurs instructions dans un fichier .py, puis vous lancez le fichier.
Idéal pour	Apprendre, tester une idée, faire un calcul rapide.	Les programmes plus longs, que l'on veut conserver et relancer.
Mémoire	Tout est perdu quand on ferme l'interpréteur.	Le fichier reste sur le disque.
Dans ce livre	Chapitres 2 à 4 : tous les exemples.	Introduit au chapitre 2, utilisé dans les TP.

C’est donc dans l’interpréteur que vous allez apprendre les notions de base : c’est le moyen le plus rapide de voir immédiatement l’effet de chaque instruction.

À retenir

- Un programme est une suite d’instructions ; un algorithme est la méthode, pensée avant le code.
- Les langages de haut niveau sont traduits pour la machine, par compilation ou par interprétation.
- Python est lisible, polyvalent, très outillé et bien documenté.
- On peut travailler avec Python en mode interactif (interpréteur) ou avec des scripts.

Exercices

Exercice 1.1 ★☆☆

Écrivez en langage courant, étape par étape, l’algorithme qui permet de préparer une tasse de thé. Soyez assez précis pour qu’une personne qui n’a jamais préparé de thé puisse le suivre.

Exercice 1.2 ★☆☆

Parmi ces langages, lesquels sont des langages de haut niveau : Python, langage machine, PHP, JavaScript, C ?

Exercice 1.3 ★☆☆

Écrivez l’algorithme qui permet de dire si un étudiant est admis, sachant qu’il est admis lorsque sa moyenne est d’au moins 10.

Exercice 1.4 ★★☆

On dispose de deux verres : l’un contient de l’eau, l’autre du jus. Écrivez l’algorithme qui permet d’échanger leur contenu. Quel élément supplémentaire vous faut-il ? Vous retrouverez cette idée au chapitre 3.

Exercice 1.5 ★☆☆

Remettez dans le bon ordre les étapes d’un projet informatique : tester le programme, définir le problème, écrire le code, concevoir l’algorithme, corriger les erreurs trouvées.

Exercice 1.6 ★★☆

Citez trois situations de votre vie courante ou de votre futur métier où un programme Python pourrait vous faire gagner du temps.

Testez-vous

1. Quelle est la différence entre un algorithme et un programme ?
2. Que veut dire « langage interprété » ? Donnez un avantage.
3. Dans quels cas préfère-t-on l’interpréteur interactif, et dans quels cas un script ?
4. Citez deux raisons d’apprendre Python.

Chapitre 2 : Installer Python et découvrir l'interpréteur de commandes

Objectifs du chapitre

- installer Python 3.15 sur votre ordinateur ;
- ouvrir l'invite de commandes et s'y déplacer avec quelques commandes ;
- lancer, utiliser et quitter l'interpréteur Python ;
- faire des calculs et afficher du texte avec `print()` ;
- lire un message d'erreur et utiliser l'aide intégrée ;
- écrire et exécuter un premier script.

2.1 Installer Python

Rendez-vous sur le site officiel **python.org**, rubrique « Downloads ». La procédure dépend de votre système.

Sous Windows

Depuis la version 3.14, Python s'installe sous Windows grâce à un outil appelé le **Python install manager**. Vous pouvez le télécharger sur python.org ou l'installer depuis l'application Microsoft Store (c'est l'approche recommandée pour la plupart des utilisateurs). Une fois l'outil installé :

- ouvrez une nouvelle fenêtre d'invite de commandes (voir la section suivante) ;
- tapez `python` : au premier lancement, l'outil peut proposer d'installer Python 3.15 et de configurer votre PATH ; acceptez ;
- si besoin, vous pouvez demander l'installation explicitement avec la commande `py install 3.15`.

Une fois installé, les commandes `python` et `py` sont disponibles dans l'invite de commandes. Les écrans exacts peuvent légèrement varier selon les mises à jour de l'outil : suivez les messages affichés.

Sous macOS et Linux

- **macOS :** téléchargez l'installeur proposé sur python.org et lancez-le.
- **Linux :** Python est en général déjà installé ; sinon, installez-le avec le gestionnaire de paquets de votre distribution.

Points d'attention

- Sous Linux et macOS, la commande s'appelle souvent `python3` au lieu de `python`. Dans ce cas, remplacez `python` par `python3` dans tous les exemples du livre.

2.2 L'invite de commandes

L'**invite de commandes** est une fenêtre où l'on donne des ordres à l'ordinateur en tapant du texte, au lieu de cliquer avec la souris. Sous Windows, elle s'appelle **cmd** ; sous Linux et macOS, on parle du **terminal**.

Pour l'ouvrir sous Windows : appuyez sur les touches Windows + R, tapez `cmd` puis validez avec Entrée. Vous obtenez une fenêtre noire qui affiche une ligne de ce type :

Invite de commandes Windows

```
C:\Users\etudiant>
```

Cette ligne indique le **dossier courant** (celui dans lequel vous vous trouvez) et attend votre commande. Voici les commandes dont vous aurez besoin :

Action	Windows (cmd)	Linux / macOS
Voir le dossier courant	`cd`	`pwd`
Lister le contenu du dossier	`dir`	`ls`
Créer un dossier	`mkdir python_cours`	`mkdir python_cours`
Entrer dans un dossier	`cd python_cours`	`cd python_cours`
Remonter d'un niveau	`cd ..`	`cd ..`
Effacer l'écran	`cls`	`clear`

À vous de jouer

- Ouvrez l'invite de commandes.
- Créez un dossier `python_cours` avec `mkdir python_cours`.
- Entrez dedans avec `cd python_cours` : c'est là que vous rangerez vos fichiers.
- Listez son contenu avec `dir` (il est vide pour l'instant).

2.3 Lancer l'interpréteur Python

Dans l'invite de commandes, commencez par vérifier que Python est bien installé :

Dans l'invite de commandes

```
C:\Users\etudiant> python --version
Python 3.15.0
```

Si la commande affiche une autre version, c'est que Python n'est pas encore bien configuré. Pour **lancer l'interpréteur**, tapez simplement `python` :

Lancement de l'interpréteur

```
C:\Users\etudiant> python
Python 3.15.0 (...) on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Les trois signes `>>>` forment l'**invite de Python** : l'interpréteur attend que vous tapiez une instruction. Chaque instruction est exécutée dès que vous appuyez sur Entrée.

Pour **quitter** l'interpréteur, tapez `exit()`. Vous pouvez aussi utiliser les touches Ctrl + Z puis Entrée sous Windows, ou Ctrl + D sous Linux et macOS. Vous revenez alors à l'invite de commandes.

Points d'attention

• Si Windows répond que `python` n'est pas reconnu, fermez la fenêtre, rouvrez-en une nouvelle (la configuration n'est prise en compte que dans une nouvelle fenêtre) et réessayez, ou essayez la commande `py`.

• Ne confondez pas les deux invites : `C:\Users\etudiant>` est celle de Windows (on y tape des commandes comme `cd`), `>>>` est celle de Python (on y tape du code Python).

2.4 Faire du calcul mental à l'ordinateur

L'interpréteur peut servir de calculatrice. Tapez les lignes suivantes (sans les `>>>`) et observez les réponses :

Dans l'interpréteur Python

```
>>> 2 + 3
5
>>> 125 + 378
503
>>> 7 * 8
56
>>> 10 / 4
2.5
>>> 10 / 5
2.0
>>> (12 + 14 + 9) / 3
11.666666666666666
```

Quelques remarques importantes :

- Les espaces autour des opérateurs sont facultatifs, mais ils rendent le code plus lisible : `2 + 3` vaut mieux que `2+3`.
- Le séparateur décimal est le **point**, pas la virgule : on écrit `3.5`.
- La division `/` donne toujours un nombre à virgule, même quand le résultat est « rond » : `10 / 5` donne `2.0`.
- Les **parenthèses** permettent de choisir l'ordre des calculs.

À vous de jouer

• Calculez le nombre de secondes dans une heure : `60 * 60`.

• Appuyez sur la flèche du haut du clavier : l'interpréteur rappelle vos instructions précédentes, ce qui évite de les retaper.

• Comparez `(12 + 14 + 9) / 3` et `12 + 14 + 9 / 3` : pourquoi les résultats sont-ils différents ?

2.5 Afficher du texte avec print()

Le texte s'écrit entre guillemets : on parle de **chaîne de caractères**. Pour afficher quelque chose à l'écran, on utilise la fonction `print()` :

Dans l'interpréteur Python

```
>>> "Bonjour"
'Bonjour'
>>> print("Bonjour")
Bonjour
>>> print("Ligne 1\nLigne 2")
Ligne 1
Ligne 2
```

Remarquez la différence : quand on tape seulement `"Bonjour"`, l'interpréteur affiche sa *représentation*, avec des apostrophes ; avec `print()`, il affiche le texte tel qu'il doit apparaître à l'utilisateur. Dans un script, seul `print()` affiche quelque chose. Quant à `\n`, c'est un caractère spécial qui provoque un retour à la ligne.

2.6 Lire un message d'erreur

Tôt ou tard, Python affichera un message d'erreur. Ne vous découragez pas : ce message vous dit précisément ce qui ne va pas. Apprenez à le lire, c'est la compétence la plus utile d'un programmeur. Voici trois erreurs très courantes :

Erreur 1 : un guillemet oublié

```
>>> print("Bonjour)
  File "<python-input-0>", line 1
    print("Bonjour)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

Erreur 2 : un nom inconnu

```
>>> nom
Traceback (most recent call last):
  File "<python-input-1>", line 1, in <module>
    nom
NameError: name 'nom' is not defined
```

Erreur 3 : une division par zéro

```
>>> 10 / 0
Traceback (most recent call last):
  File "<python-input-2>", line 1, in <module>
    10 / 0
ZeroDivisionError: division by zero
```

Pour lire une erreur, regardez d'abord la **dernière ligne** : elle donne le type d'erreur et sa description. Les lignes au-dessus indiquent l'endroit. L'aspect exact (couleurs, indications supplémentaires) peut varier légèrement selon la version de Python.

Type d'erreur	Signification	Cause fréquente
SyntaxError	Le code est mal écrit.	Guillemet ou parenthèse oublié, faute de frappe.
NameError	Un nom est utilisé sans avoir été défini.	Mot mal orthographié, texte écrit sans guillemets.
ZeroDivisionError	Division par zéro.	Un diviseur qui vaut 0.

Type d’erreur	Signification	Cause fréquente
TypeError	Une opération est faite sur des types incompatibles.	Addition d’un texte et d’un nombre (chapitre 3).

2.7 L’aide intégrée

Python contient sa propre documentation, consultable depuis l’interpréteur avec la fonction `help()`. Par exemple, pour comprendre la fonction `abs` :

Aide sur la fonction abs

```
>>> help(abs)
Help on built-in function abs in module builtins:

abs(x, /)
    Return the absolute value of the argument.
(...)
```

Si l’aide s’affiche page par page, appuyez sur la touche `q` pour revenir à l’invite. L’aide est en anglais : c’est l’occasion de vous habituer au vocabulaire technique, que vous retrouverez partout dans votre métier.

2.8 Écrire et exécuter un script

L’interpréteur oublie tout quand on le quitte. Pour conserver un programme et le relancer à volonté, on l’écrit dans un fichier texte dont le nom se termine par `.py` : c’est un **script**. Voici comment procéder :

- Ouvrez un éditeur de texte (le Bloc-notes convient, un éditeur de code comme Visual Studio Code est encore mieux).
- Tapez le code ci-dessous.
- Enregistrez le fichier sous le nom **bonjour.py**, dans votre dossier `python_cours`, avec l’encodage **UTF-8**.
- Dans l’invite de commandes, placez-vous dans le dossier avec `cd`, puis lancez le script avec `python bonjour.py`.

bonjour.py

```
# bonjour.py : mon premier script
print("Bonjour !")
print("Je programme en Python")
```

Exécution du script

```
C:\Users\etudiant\python_cours> python bonjour.py
Bonjour !
Je programme en Python
```

Points d'attention

• Sous Windows, le Bloc-notes peut enregistrer le fichier sous le nom *bonjour.py.txt*. Dans l'Explorateur, activez l'affichage des extensions de noms de fichiers pour vérifier, et choisissez « Tous les fichiers » dans la liste des types au moment d'enregistrer.

• Un script doit être lancé depuis l'**invite de commandes** (avec `python bonjour.py`), et non tapé dans l'interpréteur.

Dans le script ci-dessus, la première ligne commence par `#` : c'est un **commentaire**. Python l'ignore ; il sert à expliquer le code à ceux qui le liront, vous compris dans six mois. On peut aussi placer un commentaire à la fin d'une ligne :

```
print("Bienvenue")  # affiche un message de bienvenue
```

À retenir

• L'invite de commandes (cmd) sert à lancer des programmes ; `cd`, `dir` et `mkdir` permettent de s'y déplacer.

• On lance l'interpréteur Python avec `python` et on le quitte avec `exit()`.

• L'invite `>>>` attend une instruction Python ; elle exécute chaque ligne immédiatement.

• La dernière ligne d'un message d'erreur donne son type ; les lignes au-dessus indiquent le lieu.

• `help()` donne accès à la documentation de Python.

• Un script est un fichier `.py` lancé avec `python nom.py`.

• Les commentaires commencent par `#`.

Exercices

Exercice 2.1 ★☆☆

Dans l'interpréteur, calculez : `125 + 378` ; `12 * 15` ; `1000 / 8` ; `2 ** 10` (l'opérateur `**` est la puissance).

Pour vérifier : vous devez obtenir 503, 180, 125.0 et 1024.

Exercice 2.2 ★☆☆

Calculez le nombre de secondes dans une journée.

Pour vérifier : vous devez obtenir 86400.

Exercice 2.3 ★☆☆

Calculez `(12 + 14 + 9) / 3` puis `12 + 14 + 9 / 3`. Expliquez pourquoi les deux résultats sont différents.

Pour vérifier : 11.666666666666666 puis 29.0.

Exercice 2.4 ★☆☆

Avec `print()`, affichez les trois lignes suivantes, l'une sous l'autre : votre nom, votre filière, votre ville. Faites-le d'abord avec trois appels à `print()`, puis avec un seul appel et le caractère `\n`.

Exercice 2.5 ★★☆

Provoquez volontairement ces quatre erreurs, et notez le type d'erreur obtenu pour chacune : `print("Bonjour)` ; `print(Bonjour)` ; `10 / 0` ; `"abc" + 5`.

Pour vérifier : SyntaxError, NameError, ZeroDivisionError, TypeError.

Exercice 2.6 ★☆☆

Utilisez `help(abs)` pour lire ce que fait la fonction `abs`, puis testez `abs(-7)` et `abs(3)`.

Pour vérifier : vous devez obtenir 7 et 3.

Exercice 2.7 ★★☆

Écrivez un script `presentation.py` qui affiche votre nom, votre filière et votre ville sur trois lignes, avec un commentaire en première ligne. Lancez-le depuis l'invite de commandes.

Exercice 2.8 ★★☆

Dans l'invite de commandes, créez un dossier `exercices` à l'intérieur de `python_cours`, placez-vous dedans, puis revenez dans `python_cours` avec `cd ..`.

Testez-vous

1. Quelle est la différence entre l'invite `C:\>` et l'invite `>>>` ?
2. Comment quitte-t-on l'interpréteur Python ?
3. Quelle ligne d'un message d'erreur faut-il lire en premier, et pourquoi ?
4. Quelle est la différence entre taper `"Bonjour"` et `print("Bonjour")` dans l'interpréteur ?

Chapitre 3 : Variables et types de données

Objectifs du chapitre

- créer et utiliser des variables ;
- choisir de bons noms de variables ;
- distinguer les types int, float, str et bool ;
- convertir une valeur d'un type à un autre ;
- comprendre et éviter les erreurs liées aux types.

Lancez l'interpréteur Python (commande `python`). Tous les exemples de ce chapitre sont à taper dans l'interpréteur.

3.1 Qu'est-ce qu'une variable ?

Pour qu'un programme soit utile, il doit pouvoir **mémoriser** des informations : l'âge d'une personne, un prix, un nom... Une **variable** est un espace de la mémoire auquel on donne un nom, pour y ranger une valeur et la retrouver plus tard. On peut la comparer à une boîte étiquetée : l'étiquette est le nom de la variable, le contenu est sa valeur.

On crée une variable avec le signe `=`, appelé **affectation** :

Dans l'interpréteur Python

```
>>> age = 20
>>> age
20
>>> print(age)
20
```

Au départ, rien ne s'affiche après `age = 20` : l'interpréteur a seulement rangé la valeur 20 dans la variable `age`. Taper ensuite `age` ou `print(age)` permet de lire son contenu.

Points d'attention

- En Python, `=` ne signifie pas « égal à » comme en mathématiques. Il signifie « range la valeur de droite dans la variable de gauche ». L'instruction `age = age + 1` serait absurde en mathématiques, mais elle est parfaitement logique en programmation : elle ajoute 1 à la valeur actuelle.

Le contenu d'une variable peut changer au fil du programme :

Dans l'interpréteur Python

```
>>> age = 20
>>> age = age + 1
>>> age
21
>>> prix = 2500
>>> quantite = 4
>>> total = prix * quantite
>>> total
10000
```

Voyez comment une variable peut servir dans un calcul : Python remplace son nom par sa valeur au moment du calcul. Attention à un point important : la copie d'une valeur est indépendante.

Dans l'interpréteur Python

```
>>> a = 5
>>> b = a
>>> a = 7
>>> b
5
```

En écrivant `b = a`, on a copié la valeur 5 dans `b`. Changer ensuite `a` n'a aucun effet sur `b`.

3.2 Bien nommer ses variables

Les règles de nommage imposées par Python sont les suivantes :

- le nom contient des lettres, des chiffres et le caractère souligné `_` ;
- il ne commence pas par un chiffre ;
- il ne contient ni espace ni tiret ;
- il ne peut pas être un **mot réservé** du langage (`class`, `if`, `for`...) ;
- les majuscules et les minuscules sont distinguées : `age` et `Age` sont deux variables différentes.

Pour afficher la liste complète des mots réservés, tapez `help("keywords")` dans l'interpréteur. Au-delà des règles obligatoires, les développeurs suivent des **conventions** :

- noms en minuscules, avec des mots séparés par un souligné (style *snake_case*) : `note_python`, `prix_unitaire` ;
- noms **explicites** : `note_python` est bien meilleur que `n` ;
- les valeurs qui ne changent jamais (les constantes) s'écrivent en majuscules : `TVA = 19.25`.

À vous de jouer

• Tapez `2note = 10` : que se passe-t-il ?
• Tapez `mon-nom = 10` puis `class = 10` : même question.
• Tapez `note_2 = 10` : cette fois, aucune erreur. Pourquoi ?

3.3 Les types de données

Chaque valeur a un **type**, qui détermine ce qu'on peut faire avec elle. Python devine le type tout seul (on parle de typage dynamique). Voici les quatre types de base :

Type	Signification	Exemples
int	Nombre entier	20, -5, 0
float	Nombre à virgule (écrit avec un point)	14.5, 0.75, -3.0
str	Chaîne de caractères (texte)	"Mballa", "ETU-001"
bool	Booléen : vrai ou faux	True, False

La fonction `type()` indique le type d'une valeur :

Dans l'interpréteur Python

```
>>> type(20)
<class 'int'>
>>> type(14.5)
<class 'float'>
>>> type("Mballa")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type(None)
<class 'NoneType'>
```

Le mot `None` désigne l'**absence de valeur** ; il a son propre type. Nous l'utiliserons plus tard.

Les nombres entiers et à virgule

Les entiers (`int`) n'ont pas de limite de taille en Python. Les nombres à virgule (`float`) sont stockés avec une précision limitée, ce qui peut donner des surprises :

Dans l'interpréteur Python

```
>>> 2 ** 100
1267650600228229401496703205376
>>> 0.1 + 0.2
0.30000000000000004
>>> round(0.1 + 0.2, 2)
0.3
>>> round(3.14159, 2)
3.14
>>> abs(-7)
7
```

Le résultat de `0.1 + 0.2` n'est pas exactement 0.3 : l'ordinateur représente les nombres en binaire, et certaines décimales n'ont pas d'écriture exacte. Ce n'est pas un bug de Python : c'est vrai dans tous les langages. La fonction `round(nombre, n)` arrondit à `n` décimales ; nous l'utiliserons souvent.

Les chaînes de caractères

Une chaîne se place entre guillemets doubles ou simples. On peut assembler deux chaînes avec `+` (la **concaténation**), les répéter avec `*` et compter leurs caractères avec `len()` :

Dans l'interpréteur Python

```
>>> prenom = "Jean"
>>> nom = "Mballa"
>>> prenom + " " + nom
'Jean Mballa'
>>> "-" * 10
'----------'
>>> len(nom)
6
>>> ecole = "L'école"
>>> ecole
"L'école"
```

Le dernier exemple montre comment écrire une apostrophe : on entoure alors le texte de guillemets doubles. Pour insérer un retour à la ligne ou une tabulation dans un texte, on utilise les

caractères spéciaux `\n` et `\t` :

Dans l'interpréteur Python

```
>>> print("Nom :\tMballa\nVille :\tDouala")
Nom :	Mballa
Ville :	Douala
```

Les booléens

Un booléen ne peut prendre que deux valeurs : `True` (vrai) et `False` (faux), toujours écrites avec une majuscule. Ils sont le résultat des comparaisons, que nous verrons au chapitre 4, et sont essentiels pour prendre des décisions (partie 2).

3.4 Convertir les types

Il arrive qu'on ait besoin de changer le type d'une valeur. On utilise pour cela les fonctions `int()`, `float()` et `str()` :

Dans l'interpréteur Python

```
>>> int("12")
12
>>> float("3.5")
3.5
>>> str(10)
'10'
>>> int(7.9)
7
>>> int("12") + 8
20
>>> "12" + "8"
'128'
```

Observez bien les deux dernières lignes : avec des nombres, `+` additionne ; avec des textes, `+` colle les chaînes bout à bout. Notez aussi que `int(7.9)` **supprime** la partie décimale sans arrondir.

Le piège le plus classique est de mélanger texte et nombre :

Une erreur de type

```
>>> "10" + 5
Traceback (most recent call last):
  File "<python-input-8>", line 1, in <module>
    "10" + 5
TypeError: can only concatenate str (not "int") to str
```

Python refuse d'additionner un texte et un nombre, car il ne peut pas deviner ce que vous voulez : écrire `"105"` ou calculer 15 ? C'est à vous de convertir : `int("10") + 5` donne 15, et `"10" + str(5)` donne `"105"`.

Points d'attention

• Convertir un texte qui ne ressemble pas à un nombre, comme `int("abc")`, provoque une **ValueError**.

• Ne confondez pas `10` (le nombre) et `"10"` (le texte) : les guillemets changent le type, donc le comportement.

3.5 Affecter plusieurs variables d'un coup

Python permet de créer plusieurs variables sur une seule ligne, et d'échanger facilement le contenu de deux variables :

Dans l'interpréteur Python

```
>>> a, b = 5, 8
>>> a
5
>>> b
8
>>> a, b = b, a
>>> a
8
>>> b
5
>>> x = y = 0
>>> x
0
>>> y
0
```

L'échange `a, b = b, a` est une particularité très pratique de Python : dans beaucoup d'autres langages, il faut une troisième variable temporaire, comme le troisième verre de l'exercice 1.4.

À retenir

- Une variable est un nom associé à une valeur ; on la crée avec `=`.
- Les noms de variables sont explicites, en minuscules, avec des soulignés.
- Les types de base sont int, float, str et bool ; `type()` les identifie.
- On convertit les types avec `int()`, `float()` et `str()`.
- Mélanger texte et nombre dans une addition provoque une TypeError.
- `round(x, n)` arrondit, `len(texte)` compte les caractères.

Exercices

Exercice 3.1 ★☆☆

Parmi ces noms, lesquels sont valides en Python ? `2note`, `note_2`, `mon-nom`, `class`, `Age`, `prix ht`. Justifiez, puis vérifiez en essayant de les utiliser dans une affectation, par exemple `note_2 = 10`.

Exercice 3.2 ★☆☆

Prédisez le type de chacune de ces valeurs, puis vérifiez avec `type()` : `5`, `5.0`, `"5"`, `True`, `"True"`, `-3`, `2.5e3`.

Exercice 3.3 ★☆☆

Créez les variables `prix = 2500` et `quantite = 4`, puis calculez le total dans une variable `total`.

Pour vérifier : total doit valoir 10000.

Exercice 3.4 ★☆☆

Deux variables valent `a = 5` et `b = 8`. Échangez leurs valeurs de deux manières : d'abord avec une troisième variable, puis avec l'écriture `a, b = b, a`.

Pour vérifier : a doit valoir 8 et b doit valoir 5.

Exercice 3.5 ★★☆

Prédisez le résultat de chacune de ces expressions, puis vérifiez dans l'interpréteur : `int("42") + 8` ; `"42" + "8"` ; `float("2.5") * 2` ; `str(7) + "7"` ; `int(9.99)`.

Pour vérifier : 50, '428', 5.0, '77' et 9.

Exercice 3.6 ★☆☆

Créez la variable `mot = "Informatique"`. Affichez sa longueur, puis le mot répété deux fois, puis une ligne de tirets de la même longueur que le mot (utilisez `len()`).

Pour vérifier : la longueur est 12.

Exercice 3.7 ★★☆

Calculez `0.1 + 0.2` puis `round(0.1 + 0.2, 2)`. Que constatez-vous ? Comment expliquez-vous la différence ?

Pour vérifier : 0.30000000000000004 puis 0.3.

Exercice 3.8 ★★☆

Affichez avec `print()` la phrase suivante, apostrophes et guillemets compris : *L'étudiant dit* "*bonjour*". Cherchez comment faire (indice : on peut choisir le type de guillemets qui entoure le texte).

Exercice 3.9 ★★★

Créez les variables `nom = "Mballa"` et `age = 20`. Sans utiliser de f-string, construisez et affichez la phrase « Mballa a 20 ans » en collant des chaînes avec `+`. Quel problème rencontrez-vous, et comment le résoudre ?

Pour vérifier : il faut convertir l'âge avec str().

Testez-vous

1. Que fait exactement l'instruction `age = age + 1` ?
2. Pourquoi `mon-nom` n'est-il pas un nom de variable valide ?
3. Quelle différence entre `10` et `"10"` ?
4. Pourquoi `0.1 + 0.2` ne donne-t-il pas exactement 0.3 ?

Chapitre 4 : Opérateurs, saisie et affichage

Objectifs du chapitre

- utiliser les opérateurs arithmétiques, de comparaison et logiques ;
- demander une information à l'utilisateur avec `input()` ;
- convertir les saisies avant de les utiliser ;
- formater proprement un affichage avec les f-strings.

4.1 Les opérateurs arithmétiques

Un **opérateur** est un symbole qui effectue une opération. Voici les opérateurs arithmétiques (exemples avec 7 et 2) :

Opérateur	Rôle	Exemple	Résultat
`+`	Addition	`7 + 2`	9
`-`	Soustraction	`7 - 2`	5
`*`	Multiplication	`7 * 2`	14
`/`	Division (résultat à virgule)	`7 / 2`	3.5
`//`	Division entière (quotient)	`7 // 2`	3
`%`	Reste de la division (modulo)	`7 % 2`	1
`**`	Puissance	`7 ** 2`	49

Les deux opérateurs `//` et `%` fonctionnent ensemble. Par exemple, 17 divisé par 5 donne un quotient de 3 et un reste de 2, car 5 × 3 + 2 = 17 :

Dans l'interpréteur Python

```
>>> 17 // 5
3
>>> 17 % 5
2
>>> 17 / 5
3.4
>>> 2 ** 5
32
>>> 275 // 60
4
>>> 275 % 60
35
```

Ils sont très utiles : par exemple, 275 minutes font `275 // 60` = 4 heures et `275 % 60` = 35 minutes.

L'ordre des calculs

Comme en mathématiques, Python respecte des priorités : d'abord les parenthèses, puis la puissance, puis la multiplication, la division, la division entière et le modulo, et enfin l'addition et la soustraction. En cas de doute, ajoutez des **parenthèses** : elles ne coûtent rien et rendent le code plus clair.

Dans l'interpréteur Python

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
>>> (12 + 14 + 9) / 3
11.666666666666666
>>> 12 + 14 + 9 / 3
29.0
```

Points d'attention

• La division entière arrondit vers le bas, ce qui surprend avec les nombres négatifs : `-7 // 2` donne `-4` et non `-3`.

Les opérateurs d'affectation augmentée

Pour modifier une variable à partir de sa valeur actuelle, il existe des raccourcis : `+=`, `-=`, `*=`, `/=`, `//=`, `%=` et `**=`.

Dans l'interpréteur Python

```
>>> x = 10
>>> x += 5
>>> x
15
>>> x *= 2
>>> x
30
>>> x -= 4
>>> x
26
```

Ainsi, `x += 5` est équivalent à `x = x + 5`.

4.2 Comparer et combiner : booléens et opérateurs logiques

Une **comparaison** pose une question à laquelle Python répond par `True` ou `False` :

Opérateur	Signification	Exemple	Résultat
`==`	égal à	`5 == 5`	True
`!=`	différent de	`5 != 3`	True
`< >`	inférieur à, supérieur à	`4 < 2`	False
`<= >=`	inférieur ou égal, supérieur ou égal	`10 >= 10`	True

Dans l'interpréteur Python

```
>>> 5 == 5
True
>>> 5 == 5.0
True
>>> "a" == "A"
False
>>> "abc" < "abd"
True
>>> 10 != 10
False
```

Les textes se comparent selon l'ordre alphabétique, et les majuscules sont différentes des minuscules.

Points d'attention

• Ne confondez pas `=` (affectation : on range une valeur) et `==` (comparaison : on pose une question).

On peut combiner plusieurs comparaisons avec les opérateurs logiques `and` (les deux conditions doivent être vraies), `or` (au moins une doit être vraie) et `not` (inverse la valeur) :

Dans l'interpréteur Python

```
>>> moyenne = 12.5
>>> moyenne >= 10
True
>>> moyenne >= 10 and moyenne < 12
False
>>> moyenne >= 10 or moyenne < 5
True
>>> not (moyenne >= 10)
False
```

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

4.3 Demander une information avec input()

La fonction `input()` affiche un message, attend que l'utilisateur tape du texte puis valide avec Entrée, et **renvoie** ce texte, que l'on range dans une variable :

Dans l'interpréteur

```
>>> nom = input("Votre nom : ")
Votre nom : Awa
>>> nom
'Awa'
>>> print("Bonjour", nom)
Bonjour Awa
```

Ici, `Awa` sur la deuxième ligne est ce que l'utilisateur a tapé en réponse à la question. Essayons avec un nombre :

Le piège de input()

```
>>> age = input("Votre âge : ")
Votre âge : 20
>>> age
'20'
>>> age + 1
Traceback (most recent call last):
  File "<python-input-5>", line 1, in <module>
    age + 1
TypeError: can only concatenate str (not "int") to str
>>> int(age) + 1
21
```

Points d'attention

• **input() renvoie toujours une chaîne de caractères**, même si l'utilisateur tape un nombre. Remarquez les apostrophes autour de `'20'` : c'est un texte. Pour faire un calcul, il faut convertir la saisie avec `int()` ou `float()`.

• On peut combiner saisie et conversion en une seule ligne : `age = int(input("Votre âge : "))`.

Si l'utilisateur tape un texte qui n'est pas un nombre alors que vous attendez un nombre, Python lève une **ValueError**. La gestion de ce cas fait l'objet du chapitre sur les exceptions.

4.4 Afficher avec élégance

La fonction `print()` accepte plusieurs valeurs séparées par des virgules, et deux options utiles : `sep` (ce qui sépare les valeurs) et `end` (ce qui est écrit à la fin).

Les options sep et end

```
>>> print("a", "b", "c")
a b c
>>> print("a", "b", "c", sep="-")
a-b-c
>>> print("Début", end=" ... ")
Début ... >>> print("fin")
fin
```

Par défaut, `print()` termine par un retour à la ligne, et les valeurs sont séparées par un espace. Avec `end=" ... "`, le curseur reste sur la même ligne, ce qui explique pourquoi l'invite `>>>` apparaît juste après le texte dans l'exemple ci-dessus.

Les f-strings

La méthode la plus pratique pour construire un texte contenant des variables est la **f-string** (*formatted string*) : on place un `f` juste devant les guillemets, et on met les variables ou les calculs entre accolades `{ }` :

Dans l'interpréteur Python

```
>>> nom = "Mballa"
>>> age = 20
>>> print(f"{nom} a {age} ans")
Mballa a 20 ans
>>> print(f"Dans 10 ans, {nom} aura {age + 10} ans")
Dans 10 ans, Mballa aura 30 ans
```

On peut régler la présentation des nombres en ajoutant un **format** après les deux points : `:.2f` affiche deux chiffres après la virgule, `:,` ajoute un séparateur de milliers.

Dans l'interpréteur Python

```
>>> moyenne = 13.3333333
>>> print(f"Moyenne : {moyenne:.2f} / 20")
Moyenne : 13.33 / 20
>>> montant = 1234567.891
>>> print(f"{montant:,.2f}")
1,234,567.89
```

Aligner des colonnes

Pour afficher des tableaux bien alignés, on précise une **largeur** : `<15` aligne à gauche sur 15 caractères, `>8` aligne à droite sur 8 caractères, `^10` centre sur 10 caractères. On peut combiner : `>6.2f` signifie « largeur 6, deux décimales, aligné à droite ».

Dans l'interpréteur Python

```
>>> matiere = "Python"
>>> note = 15.5
>>> print(f"{matiere:<15}{note:>6.2f}")
Python          15.50
>>> print(f"{'Algo':<15}{12:>6.2f}")
Algo            12.00
```

Retenez la forme générale d'un champ : `{valeur:alignement largeur .décimales type}`. Vous la retrouverez dans tous les affichages en colonnes, et dans les TP.

À retenir

- Opérateurs arithmétiques : `+ - * / // % **` ; priorité aux parenthèses.
- `//` donne le quotient entier, `%` donne le reste.
- Les comparaisons (`== != < > <= >=`) produisent un booléen ; `and`, `or` et `not` les combinent.
- `=` range une valeur, `==` compare deux valeurs.
- `input()` renvoie toujours une chaîne : on convertit avec `int()` ou `float()`.
- Les f-strings (`f"{variable}"`) assemblent texte et valeurs, avec des formats pour les décimales et l'alignement.

Exercices

Exercice 4.1 ★☆☆

Calculez dans l'interpréteur : `17 // 5`, `17 % 5`, `17 / 5`, `2 ** 5`, `-7 // 2`.

Pour vérifier : 3, 2, 3.4, 32 et -4.

Exercice 4.2 ★☆☆

Prédisez puis vérifiez : `2 + 3 * 4` et `(2 + 3) * 4`.

Pour vérifier : 14 et 20.

Exercice 4.3 ★★☆

Un cours dure 275 minutes. À l'aide de variables et des opérateurs `//` et `%`, affichez cette durée en heures et minutes avec une phrase du type « 275 minutes = 4 h 35 min ».

Pour vérifier : 4 h 35 min.

Exercice 4.4 ★★☆

Prédisez le résultat de chacune de ces expressions, puis vérifiez : `5 > 3` ; `5 == 5.0` ; `"a" == "A"` ; `not (4 > 2)` ; `3 < 5 and 5 < 4` ; `3 < 5 or 5 < 4` ; `10 != 10`.

Pour vérifier : True, True, False, False, False, True, False.

Exercice 4.5 ★★☆

Créez `x = 10` puis exécutez dans cet ordre : `x += 5`, `x *= 2`, `x -= 4`, `x //= 3`. Prédisez la valeur finale avant de la vérifier.

Pour vérifier : x vaut 8.

Exercice 4.6 ★★☆

Écrivez un script `avenir.py` qui demande le nom et l'âge de l'utilisateur, puis affiche : « Dans 10 ans, Awa aura 30 ans. » (avec les valeurs saisies).

Exercice 4.7 ★★☆

Écrivez un script `rectangle.py` qui demande la longueur et la largeur d'un rectangle (nombres à virgule), puis affiche son aire et son périmètre avec deux décimales.

Pour vérifier : pour 5.5 et 3 : aire 16.50 et périmètre 17.00.

Exercice 4.8 ★★☆

À l'aide d'une f-string, affichez le nombre 1234567.891 avec un séparateur de milliers et deux décimales.

Pour vérifier : 1,234,567.89.

Exercice 4.9 ★★★

Écrivez un script `conversion.py` qui demande un prix hors taxe et un taux de TVA en pourcentage, puis affiche le montant de la TVA et le prix TTC avec deux décimales, alignés sur deux lignes en colonnes.

Pour vérifier : pour 10000 et 19.25 : TVA 1925.00 et prix TTC 11925.00.

Exercice 4.10 ★★★

Écrivez un script qui demande un nombre de secondes et l'affiche en jours, heures, minutes et secondes.

Pour vérifier : pour 100000 secondes : 1 jour, 3 heures, 46 minutes et 40 secondes.

Testez-vous

1. Que renvoient `//` et `%` ? Donnez un exemple.
2. Quelle est la différence entre `=` et `==` ?
3. Pourquoi `input()` nécessite-t-elle souvent une conversion ?
4. Que signifie le format `:>8.2f` dans une f-string ?

Travaux pratiques de la Partie 1

Les trois TP qui suivent réunissent toutes les notions de la Partie 1 : variables, opérateurs, saisie avec `input()`, conversions et affichage avec les f-strings. Pour chacun :

- écrivez votre programme dans un **script** `.py` enregistré dans votre dossier `python_cours` (encodage UTF-8) ;
- lancez-le depuis l'invite de commandes avec `python nom_du_script.py` ;
- avancez **étape par étape** : testez chaque étape avant de passer à la suivante ;
- comparez votre affichage à l'**exemple d'exécution** donné (les valeurs en gras dans les exemples sont celles saisies par l'utilisateur) ;
- n'hésitez pas à revenir aux chapitres et à utiliser `help()`.

> **Rappel de méthode**
> 1. Lisez l'énoncé en entier. 2. Écrivez sur papier les données, les calculs et l'affichage attendu (c'est votre algorithme). 3. Traduisez en Python étape par étape. 4. Testez avec les valeurs de l'exemple, puis avec d'autres valeurs.

TP 1 : Un convertisseur de durées, de températures et de monnaies

Compétences mobilisées

Saisie avec `input()`, conversions `int()` et `float()`, opérateurs `//` et `%`, f-strings.

Durée estimée : 1 h 30.

Énoncé

Vous devez écrire un script **convertisseur.py** qui propose successivement trois conversions. Le programme les enchaîne l'une après l'autre, sans menu.

Étape 1 : convertir des secondes en heures, minutes et secondes

Le programme demande un nombre de secondes (un entier) et l'affiche sous la forme heures, minutes, secondes.

Exemple d'exécution

```
Nombre de secondes : 3725
3725 secondes = 1 h 2 min 5 s
```

Indice : il y a 3600 secondes dans une heure. Calculez d'abord les heures avec `//`, puis travaillez sur le reste obtenu avec `%` pour les minutes et les secondes.

Étape 2 : convertir des degrés Celsius en degrés Fahrenheit

La formule est : Fahrenheit = Celsius × 9 / 5 + 32. Le programme demande une température en degrés Celsius (un nombre à virgule) et affiche l'équivalent en degrés Fahrenheit.

Exemple d'exécution

```
Température en degrés Celsius : 25
25.0 °C = 77.0 °F
```

Étape 3 : convertir des francs CFA en euros

Le franc CFA est lié à l'euro par une parité fixe : 1 euro = 655,957 francs CFA. Le programme demande un montant en francs CFA et l'affiche en euros, avec deux décimales. Stockez la parité dans une variable en majuscules, par exemple `PARITE`, plutôt que d'écrire le nombre directement dans le calcul.

Exemple d'exécution

```
Montant en francs CFA : 100000
100000.0 FCFA = 152.45 euros
```

Pour aller plus loin (facultatif)

- Ajoutez la conversion inverse : euros vers francs CFA.
- Affichez un titre encadré au début du programme avec les caractères `=` répétés, comme dans l'exemple du chapitre 2.

- Après la Partie 2, vous pourrez proposer un menu pour choisir la conversion.

Critères de réussite

- Le script s’exécute sans erreur et donne les résultats des exemples.
- Les messages affichés à l’utilisateur sont clairs.
- Les noms de variables sont explicites et la parité est une constante.
- Le code est commenté (un commentaire par étape).

TP 2 : Un ticket de caisse

Compétences mobilisées

Saisie et conversion de plusieurs données, calculs en chaîne, pourcentages, mise en forme alignée avec les f-strings.

Durée estimée : 2 h.

Énoncé

Une petite boutique vous demande un programme qui édite le ticket de caisse d'un client qui achète **deux articles**. Écrivez le script **ticket.py**.

Étape 1 : la saisie

Le programme demande, pour chacun des deux articles : le nom, le prix unitaire (nombre à virgule) et la quantité (entier). Il demande ensuite le taux de remise en pourcentage, puis le taux de TVA en pourcentage.

Étape 2 : les calculs

- le total de chaque ligne : prix unitaire × quantité ;
- le sous-total : somme des totaux des deux lignes ;
- le montant de la remise : sous-total × taux de remise / 100 ;
- le net HT : sous-total moins la remise ;
- le montant de la TVA : net HT × taux de TVA / 100 ;
- le total TTC : net HT plus TVA.

Étape 3 : l'affichage

Reproduisez la présentation de l'exemple ci-dessous : le nom de l'article aligné à gauche sur 12 caractères, la quantité sur 3, le prix unitaire sur 9 et le total de ligne sur 10, tous avec deux décimales. Les lignes de résumé affichent un libellé aligné à gauche sur 30 caractères, suivi d'un montant sur 10. Le titre est centré sur 40 caractères et entouré de signes `=`.

Exemple d'exécution

```
Article 1 : Cahier
Prix unitaire : 500
Quantité : 4
Article 2 : Stylo
Prix unitaire : 250
Quantité : 6
Taux de remise en % : 5
Taux de TVA en % : 19.25

========== TICKET DE CAISSE ==========
Cahier        4 x    500.00 =    2000.00
Stylo         6 x    250.00 =    1500.00
---------------------------------------
Sous-total                       3500.00
Remise (5.0 %)                   -175.00
Net HT                           3325.00
TVA (19.25 %)                     640.06
TOTAL TTC                        3965.06
=======================================
```

Indices : convertissez les saisies numériques avec `float()` ou `int()` dès la lecture ; pour centrer le titre avec des `=` de chaque côté, cherchez comment indiquer un **caractère de remplissage** avant le symbole `^` dans une f-string ; le nom et la quantité des articles sont affichés tels que saisis.

Pour aller plus loin (facultatif)

- Ajoutez un troisième article.
- Affichez aussi le nombre total d'articles achetés.
- Après la Partie 2, refusez une quantité négative ou nulle.

Critères de réussite

- Votre ticket est identique à l'exemple pour les mêmes valeurs saisies.
- Avec d'autres valeurs (par exemple une remise de 0 %), le ticket reste correct.
- Les calculs intermédiaires sont rangés dans des variables bien nommées.

TP 3 : Le bulletin de notes d'un étudiant

Compétences mobilisées

Constantes, moyenne pondérée, saisie et conversion de plusieurs notes, tableau aligné avec les f-strings.

Durée estimée : 2 h.

Énoncé

Le secrétariat de votre établissement veut un petit programme qui calcule la moyenne pondérée d'un étudiant et édite son bulletin. Écrivez le script **bulletin.py**.

Les données

L'étudiant est évalué dans quatre matières, chacune avec son coefficient :

Matière	Coefficient
Python	4
Algorithmique	3
Bases de données	3
Anglais technique	2

Les coefficients ne sont pas saisis : rangez-les dans des variables en majuscules en début de script. Le programme demande le nom de l'étudiant puis ses quatre notes sur 20 (nombres à virgule).

Les calculs

- les **points** de chaque matière : note × coefficient ;
- le total des coefficients et le total des points ;
- la **moyenne générale** : total des points divisé par le total des coefficients.

L'affichage

Le bulletin est un tableau de 42 caractères de large : matière alignée à gauche sur 20, note sur 6, coefficient sur 6 et points sur 10. Reproduisez l'exemple suivant :

Exemple d'exécution

```
Nom de l'étudiant : Mballa Jean
Note de Python : 15
Note d'algorithmique : 12
Note de bases de données : 14
Note d'anglais technique : 10

=========== BULLETIN DE NOTES ============
Étudiant : Mballa Jean

Matière               Note  Coef    Points
-----------------------------------------
Python               15.00     4     60.00
Algorithmique        12.00     3     36.00
Bases de données     14.00     3     42.00
Anglais technique    10.00     2     20.00
-----------------------------------------
Total                           12    158.00
Moyenne générale : 13.17 / 20
```

Indices : pour la ligne « Total », vous pouvez afficher une chaîne vide dans la colonne des notes en lui donnant la même largeur de 6 caractères ; arrondissez la moyenne à deux décimales seulement **à l'affichage**, pas dans le calcul.

Pour aller plus loin (facultatif)

- Ajoutez une cinquième matière de votre choix.
- Affichez la moyenne en pourcentage en plus de la note sur 20.
- Après la Partie 2, ajoutez la mention (admis ou ajourné, puis passable, assez bien, bien...) selon la moyenne.

Critères de réussite

- Votre bulletin est identique à l'exemple pour les mêmes valeurs saisies.
- Changer un coefficient en début de script modifie le résultat sans toucher aux calculs.
- Le code est organisé en trois blocs commentés : saisie, calculs, affichage.

Lexique et fiche mémo de la Partie 1

Lexique

Terme	Définition
Algorithme	Méthode décrite étape par étape pour résoudre un problème.
Programme	Suite d'instructions écrites dans un langage de programmation.
Interpréteur	Programme qui lit et exécute le code Python instruction par instruction.
Invite de commandes	Fenêtre où l'on tape des commandes (cmd sous Windows, terminal sous Linux et macOS).
Invite Python	Les trois signes >>> qui indiquent que l'interpréteur attend une instruction.
Script	Fichier .py contenant un programme, lancé avec python nom.py.
Instruction	Ordre donné à l'ordinateur, par exemple un appel à print().
Expression	Combinaison de valeurs et d'opérateurs qui produit un résultat.
Variable	Nom associé à une valeur gardée en mémoire.
Affectation	Action de ranger une valeur dans une variable avec le signe =.
Type	Nature d'une valeur : int, float, str, bool...
Entier (int)	Nombre sans partie décimale.
Flottant (float)	Nombre à virgule, écrit avec un point.
Chaîne (str)	Texte placé entre guillemets.
Booléen (bool)	Valeur logique : True ou False.
Opérateur	Symbole qui effectue une opération (+, -, *, /, ==, and...).
Fonction	Instruction nommée qui effectue une tâche, comme print() ou len().
Argument	Valeur placée entre les parenthèses d'une fonction.
Commentaire	Texte précédé de # ignoré par Python.
Conversion	Changement du type d'une valeur avec int(), float() ou str().
f-string	Chaîne précédée de f qui permet d'insérer des valeurs entre accolades.
Erreur / exception	Message affiché quand Python ne peut pas exécuter une instruction.

Fiche mémo

Pour...	Je tape...
Vérifier la version de Python	`python --version`
Lancer l'interpréteur	`python`
Quitter l'interpréteur	`exit()`
Lancer un script	`python nom.py`
Afficher la documentation d'une fonction	`help(print)`

Pour…	Je tape…
Connaître le type d’une valeur	`type(valeur)`
Convertir en entier / nombre à virgule / texte	`int()` / `float()` / `str()`
Arrondir à 2 décimales	`round(x, 2)`
Demander une saisie	`input("question")`
Afficher des valeurs	`print(a, b)` ou `print(f"{a} {b}")`
Quotient et reste d’une division	`a // b` et `a % b`
Afficher avec 2 décimales, aligné à droite sur 8	`f"{x:>8.2f}"`